

Estudo Dirigido · Guia 2

Surface e Sprites

Atualização da Tela e Manipulação de Imagens

Este guia é a continuação do Guia 1 (Configuração Inicial) e aprofunda dois conceitos fundamentais: a atualização eficiente da Surface a cada frame e o uso de Sprites para representar visualmente os objetos do jogo.

Objetivos de Aprendizagem

- Entender o ciclo de renderização: limpar → desenhar → exibir
- Carregar imagens externas com `pygame.image.load()`
- Usar `blit()` para posicionar sprites na tela
- Tratar erros de carregamento com imagem de fallback
- Calcular posições dinâmicas (centro, fundo da tela)

1. Revisão: O Ciclo de Renderização

Antes de avançar, vamos consolidar como o Pygame atualiza a tela a cada frame. O processo segue três etapas obrigatórias dentro do Game Loop:

1. LIMPAR <code>SCREEN.fill(COR)</code> Apaga o frame anterior para evitar rastros	2. DESENHAR <code>SCREEN.blit(...)</code> Posiciona sprites e elementos na tela	3. EXIBIR <code>display.flip()</code> Publica o frame desenhado para o usuário
---	--	---

⚠ Por que limpar antes de desenhar?

Se você não chamar `SCREEN.fill()` no início de cada frame, os sprites anteriores ficam visíveis na tela — criando um efeito de rastro indesejado à medida que os objetos se movem.

1.1. Game Loop com Renderização Completa

O esqueleto abaixo integra as três etapas do ciclo de renderização:

```
BACKGROUND_COLOR = (0, 0, 0) # Preto
running = True
```

```
while running:

    # 1. Eventos
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    # 2. Atualização do estado (lógica do jogo)
    # ... movimentação, colisões, pontuação ...

    # 3. Renderização
    SCREEN.fill(BACKGROUND_COLOR)    # 3a. Limpa o frame
    SCREEN.blit(...)                  # 3b. Desenha sprites (ver seção 2)
    pygame.display.flip()             # 3c. Exibe o novo frame

pygame.quit()
sys.exit()
```

2. Sprites: Carregando e Desenhando Imagens

Um Sprite é qualquer elemento visual do jogo — jogador, inimigo, projétil, item. No Pygame, sprites são representados por objetos Surface carregados a partir de arquivos de imagem.

2.1. Carregando uma Imagem

Use `pygame.image.load()` para carregar um arquivo de imagem do disco. Sempre chame `.convert_alpha()` em seguida para otimizar o processamento e preservar a transparência (canal alfa).

✗ Sem otimização

```
img = pygame.image.load("nave.png")
```

Mais lento — Pygame converte o formato a cada blit.

✓ Com otimização

```
img = pygame.image.load("nave.png")
    .convert_alpha()
```

Mais rápido — pré-converte para o formato de tela.

2.2. Tratamento de Erros (Fallback)

Arquivos de imagem podem estar ausentes ou corrompidos. Sempre envolva o carregamento em `try/except` para evitar que o jogo trave — e forneça uma imagem de substituição funcional:

```
try:
    PLAYER_IMAGE = pygame.image.load("nave.png").convert_alpha()
```

```
except pygame.error as e:
    print(f"Erro ao carregar imagem: {e}")
    # Fallback: cria uma Surface de 50x50 pixels vermelha
    PLAYER_IMAGE = pygame.Surface((50, 50))
    PLAYER_IMAGE.fill((255, 0, 0))
```

Boas práticas de assets

Crie uma pasta `assets/` na raiz do projeto e armazene todas as imagens lá. Use `pygame.image.load('assets/nave.png')` para caminhos relativos consistentes em qualquer sistema operacional.

2.3. Posicionando o Sprite na Tela

Após carregar a imagem, calcule as coordenadas onde ela deve aparecer. Use `get_width()` e `get_height()` para calcular posições dinâmicas relativas ao tamanho da janela:

```
# Centro horizontal, perto do fundo da tela
player_x = (SCREEN_WIDTH / 2) - (PLAYER_IMAGE.get_width() / 2)
player_y = SCREEN_HEIGHT - PLAYER_IMAGE.get_height() - 10

# Outras posições úteis:
# Centro da tela:
# x = (SCREEN_WIDTH - img.get_width()) // 2
# y = (SCREEN_HEIGHT - img.get_height()) // 2

# Canto superior esquerdo:
# x, y = 0, 0
```

2.4. Desenhando com `blit()`

O método `blit()` (Block Image Transfer) copia os pixels de uma `Surface` sobre outra. É a operação fundamental de renderização no Pygame:

```
# Dentro do Game Loop, na etapa 3b (Desenhar):
# SCREEN.blit(surface, (x, y))

SCREEN.blit(PLAYER_IMAGE, (player_x, player_y))

# Você pode desenhar vários sprites em sequência.
# A ORDEM importa: o último desenhado fica na frente.
SCREEN.blit(BACKGROUND_IMAGE, (0, 0)) # fundo (primeiro)
SCREEN.blit(ENEMY_IMAGE, (ex, ey)) # inimigo
SCREEN.blit(PLAYER_IMAGE, (px, py)) # jogador (frente)
```

💡 Ordem de desenho (Z-order)

O Pygame não tem camadas automáticas — você controla a profundidade desenhando na ordem certa: fundo primeiro, objetos em cima, HUD (placar, vida) por último.

3. Código Completo: Sprite em Movimento

O exemplo abaixo une todos os conceitos: inicialização, carregamento de sprite com fallback, posicionamento dinâmico e game loop com as três etapas de renderização:

```
import pygame
import sys

# — Inicialização —
pygame.init()
SCREEN_WIDTH, SCREEN_HEIGHT = 800, 600
SCREEN = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))
pygame.display.set_caption("Jogo com Sprite")
clock = pygame.time.Clock()

# — Carregamento do sprite —
try:
    PLAYER_IMAGE = pygame.image.load("nave.png").convert_alpha()
except pygame.error:
    PLAYER_IMAGE = pygame.Surface((50, 50))
    PLAYER_IMAGE.fill((0, 180, 255)) # azul como fallback

# — Posição inicial —
player_x = (SCREEN_WIDTH // 2) - (PLAYER_IMAGE.get_width() // 2)
player_y = SCREEN_HEIGHT - PLAYER_IMAGE.get_height() - 10
velocidade = 5

# — Cores —
PRETO = (0, 0, 0)

# — Game Loop —
running = True
while running:

    # 1. Eventos
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
```

```

# 2. Movimentação com teclas de seta
teclas = pygame.key.get_pressed()
if teclas[pygame.K_LEFT] and player_x > 0:
    player_x -= velocidade
if teclas[pygame.K_RIGHT] and player_x < SCREEN_WIDTH -
PLAYER_IMAGE.get_width():
    player_x += velocidade

# 3. Renderização
SCREEN.fill(PRETO) # 3a. Limpa
SCREEN.blit(PLAYER_IMAGE, (player_x, player_y)) # 3b. Desenha
pygame.display.flip() # 3c. Exibe
clock.tick(60) # 60 FPS

pygame.quit()
sys.exit()

```

4. Referência de Métodos

Método / Função	Descrição
<code>pygame.image.load(path)</code>	Carrega uma imagem do disco e retorna uma Surface
<code>.convert_alpha()</code>	Otimiza a Surface para renderização, preservando transparência
<code>Surface.get_width()</code>	Retorna a largura da Surface em pixels
<code>Surface.get_height()</code>	Retorna a altura da Surface em pixels
<code>Surface.get_size()</code>	Retorna uma tupla (largura, altura) da Surface
<code>SCREEN.blit(src, (x,y))</code>	Copia a Surface src para SCREEN na posição (x, y)
<code>SCREEN.fill(cor)</code>	Preenche toda a Surface com uma cor RGB sólida
<code>pygame.display.flip()</code>	Atualiza a janela inteira (exibe o frame desenhado)
<code>pygame.key.get_pressed()</code>	Retorna o estado de todas as teclas (pressionadas ou não)

5. Exercícios Práticos

Coloque em prática os conceitos com os desafios abaixo:

Exercício 1 — Fundo colorido

Mude `BACKGROUND_COLOR` para `(20, 20, 60)` (azul noturno). Observe o contraste com o sprite.

Exercício 2 — Fallback personalizado

Crie um fallback com `pygame.Surface` e desenhe um triângulo verde nela usando `pygame.draw.polygon()`.

Exercício 3 — Dois sprites

Carregue duas imagens distintas (ou dois fallbacks com cores diferentes) e desenhe ambas na tela simultaneamente.

Exercício 4 — Movimento vertical

Adicione suporte às teclas `K_UP` e `K_DOWN` para mover o sprite verticalmente, com limites nas bordas da tela.

Exercício 5 — Rastro proposital

Comente a linha `SCREEN.fill()` e execute. Observe o efeito de rastro. Depois reative e compare o resultado.

6. Próximos Passos

Com Surface e Sprites dominados, você está pronto para avançar para os seguintes tópicos:

Guia 3 — Colisões

- `pygame.Rect` e detecção de colisão
- `colliderect()` e `collidelist()`
- Grupos de sprites (`pygame.sprite.Group`)

Guia 4 — Sons e Fontes

- `pygame.mixer` para efeitos sonoros
- `pygame.font` para texto na tela
- Placar e HUD do jogo